



INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Internacional de Ingeniería
y Ciencias Sociales y Administrativas

Licenciatura en Ciencias de la informática

Asignatura:
Abstracción y uso de datos

Reporte de elaboración de códigos
(ordenamientos y ADT en C++)

Secuencia: 3CM30

ALUMNAS:
Hernández Rodríguez Jessica

Profesor:
Entzana Garduño Yventz



16 PE. Ordenamiento Burbuja (Programación Estructurada)

Archivo: 16 ordenamiento burbuja PE.cpp

Concepto principal: Algoritmos de ordenamiento: burbuja PE

Descripción:

Se implementa el algoritmo de ordenamiento Burbuja en el paradigma estructurado usando structs. Se define la estructura Persona con campos nombre (string) y edad (int). La función burbuja() ordena un arreglo de Personas de forma ascendente por edad. La función mostrar() imprime nombre y edad de cada elemento. En main() el usuario ingresa la cantidad de personas, sus datos, y se muestra el resultado ordenado.

Técnicas utilizadas:

Definición de struct como tipo de dato compuesto, paso de arreglo de structs a funciones, doble ciclo for para comparaciones adyacentes, intercambio con variable temporal de tipo struct, función auxiliar de visualización separada de la lógica de ordenamiento.

Fragmento clave:

```
void burbuja(Persona arr[], int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++){
            if(arr[j].edad > arr[j+1].edad){
                Persona temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

Análisis:

El ordenamiento Burbuja en PE separa claramente la lógica del algoritmo en funciones libres. Con complejidad $O(n^2)$, es el menos eficiente de los tres algoritmos implementados en el parcial, pero su comprensión es fundamental como base. El intercambio de structs completos ilustra que en C++ los tipos compuestos se pueden operar igual que los tipos primitivos cuando se asignan con el operador =.

16 POO. Ordenamiento Burbuja (Programación Orientada a Objetos)

Archivo: 16 ordenamiento burbuja POO.cpp

Concepto principal: Algoritmos de ordenamiento: burbuja POO

Descripción:

Se reescribe el algoritmo Burbuja bajo el paradigma orientado a objetos. La clase Persona encapsula nombre y edad como atributos públicos. La clase Burbuja agrupa el comportamiento del algoritmo en dos métodos: ordenar, que ejecuta las comparaciones e intercambios, y mostrar, que recorre e imprime el arreglo. En main se instancia un objeto Burbuja y se invocan sus métodos.

Técnicas utilizadas:

Encapsulación del algoritmo en una clase con métodos cohesivos, separación entre clase de datos (Persona) y clase de comportamiento (Burbuja), instanciación de objeto y llamada a métodos mediante el operador punto (.), polimorfismo estructural.

Fragmento clave:

```
class Burbuja{
public:
    void ordenar(Persona arr[], int n){
        for(int i=0;i<n-1;i++){
            for(int j=0;j<n-1-i;j++){
                if(arr[j].edad > arr[j+1].edad){
                    Persona temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
    }
};
```

Análisis:

La versión POO del Burbuja demuestra la ventaja del encapsulamiento: toda la responsabilidad del ordenamiento reside en la clase Burbuja, haciéndola reutilizable. Comparada con la versión PE, el algoritmo es idéntico, pero el diseño favorece la cohesión y reduce el acoplamiento entre la lógica de ordenamiento y el programa principal.

17 PE. Ordenamiento Merge Sort (Programación Estructurada)

Archivo: 17 ordenamiento merge sort PE.cpp

Concepto principal: Algoritmos de ordenamiento: merge Sort PE

Descripción:

Se implementa el algoritmo Merge Sort mediante dos funciones libres: merge y mergeSort. La función mergeSort divide recursivamente el arreglo en mitades hasta obtener subarreglos de un solo elemento. La función merge combina dos subarreglos ordenados usando un arreglo temporal de tamaño 100, comparando el campo edad de cada Persona y copiando el resultado al arreglo original.

Técnicas utilizadas:

Paradigma de divide y conquista, recursividad con dos llamadas por nivel, uso de arreglo auxiliar temporal para la fusión, punteros de índice i, j, k para controlar la fusión, copia de residuos con ciclos while independientes.

Fragmento clave:

```
void mergeSort(Persona arr[], int ini, int fin){
    if(ini>=fin) return;
    int mid=(ini+fin)/2;
    mergeSort(arr,ini,mid);
    mergeSort(arr,mid+1,fin);
    merge(arr,ini,mid,fin);
}
```

Análisis:

Merge Sort garantiza $O(n \log n)$ en todos los casos, lo que lo hace predecible y eficiente frente a Burbuja. La recursión divide el problema en subproblemas de menor tamaño que se resuelven

independientemente, un patrón fundamental en algoritmos avanzados. El uso de un arreglo temporal en merge() implica un costo adicional de memoria $O(n)$, que es el compromiso clásico de este algoritmo.

17 POO. Ordenamiento Merge Sort (Programación Orientada a Objetos)

Archivo: 17 ordenamiento merge sort POO.cpp

Concepto principal: Algoritmos de ordenamiento: merge Sort POO

Descripción:

El algoritmo Merge Sort se encapsula en la clase MergeSort, que expone tres métodos públicos: merge para la fusión de subarreglos, ordenar para la división recursiva, y mostrar para la presentación del resultado. La clase Persona define los datos. En main se instancia MergeSort y se llama a ordenar con los índices inicial 0 y final n-1.

Técnicas utilizadas:

Encapsulación de un algoritmo recursivo complejo en una clase con responsabilidades bien definidas, separación entre método de fusión y método de división, método de presentación independiente, diseño orientado a objetos aplicado a algoritmos de alto rendimiento.

Fragmento clave:

```
class MergeSort{
public:
    void merge(Persona arr[], int ini, int mid, int fin){ /*...*/ }

    void ordenar(Persona arr[], int ini, int fin){
        if(ini>=fin) return;
        int mid=(ini+fin)/2;
        ordenar(arr,ini,mid);
        ordenar(arr,mid+1,fin);
        merge(arr,ini,mid,fin);
    }

    void mostrar(Persona arr[], int n){ /*...*/ }
};
```

Análisis:

La clase MergeSort ilustra cómo algoritmos con múltiples fases (dividir, fusionar, mostrar) se benefician del encapsulamiento en POO. Cada método tiene una única responsabilidad, siguiendo el principio SRP. La recursión dentro de un método de instancia funciona igual que en una función libre, ya que el objeto no almacena estado durante la recursión.

18 PE. Ordenamiento Quick Sort (Programación Estructurada)

Archivo: 18 ordenamiento quick sort PE.cpp

Concepto principal: Algoritmos de ordenamiento: quick Sort PE

Descripción:

Se implementa el algoritmo Quick Sort en PE mediante dos funciones libres: particion y quickSort. La función particion utiliza el esquema de Lomuto, seleccionando el último elemento como pivote.

Reordena el arreglo colocando los elementos menores al pivote a la izquierda, luego ubica el pivote en su posición definitiva y retorna su índice. quickSort se llama recursivamente en los dos sub-arreglos generados.

Técnicas utilizadas:

Esquema de partición de Lomuto con pivote en el último elemento, índice i como marcador de la frontera entre menores y mayores, recursión en sub-arreglos izquierdo (ini a pi-1) y derecho (pi+1 a fin), intercambio in-place sin arreglo auxiliar.

Fragmento clave:

```
int particion(Persona arr[], int ini, int fin){
    int pivote=arr[fin].edad;
    int i=ini-1;
    for(int j=ini;j<fin;j++){
        if(arr[j].edad < pivote){
            i++;
            Persona temp=arr[i]; arr[i]=arr[j]; arr[j]=temp;
        }
    }
    Persona temp=arr[i+1]; arr[i+1]=arr[fin]; arr[fin]=temp;
    return i+1;
}
```

Análisis:

Quick Sort es el algoritmo de ordenamiento más utilizado en la práctica. En el caso promedio opera en $O(n \log n)$ con muy bajo costo de memoria ($O(\log n)$ por la pila de recursión), ya que ordena in-place. Sin embargo, en el peor caso (arreglo ya ordenado con pivote en el extremo) degrada a $O(n^2)$. La selección del pivote es el factor crítico de su rendimiento.

18 POO. Ordenamiento Quick Sort (Programación Orientada a Objetos)

Archivo: 18 ORDENAMIENTO POO.cpp

Concepto principal: Algoritmos de ordenamiento: quick Sort POO

Descripción:

El algoritmo Quick Sort se encapsula en la clase QuickSort, que define tres métodos públicos: particion que aplica el esquema de Lomuto y retorna el índice del pivote final, ordenar que realiza la recursión en los sub-arreglos, y mostrar que imprime el resultado. La clase Persona almacena los datos. En main se instancia QuickSort y se invoca ordenar con índices 0 y n-1.

Técnicas utilizadas:

Encapsulación de la partición y la recursión en métodos de clase, método particion con retorno de entero indicando posición del pivote, llamadas recursivas a método propio (this->ordenar()), polimorfismo estructural entre clase de datos y clase de algoritmo.

Fragmento clave:

```
class QuickSort{
public:
    int particion(Persona arr[], int ini, int fin){
        int pivote=arr[fin].edad; int i=ini-1;
        for(int j=ini;j<fin;j++){
            if(arr[j].edad < pivote){ i++; /*swap*/; }
        }
        /*swap arr[i+1] y arr[fin]*/;
        return i+1;
    }
}
```

```

void ordenar(Persona arr[], int ini, int fin){
    if(ini<fin){
        int pi=particion(arr,ini,fin);
        ordenar(arr,ini,pi-1);
        ordenar(arr,pi+1,fin);
    }
};

```

Análisis:

La versión POO del Quick Sort demuestra que incluso algoritmos con múltiples fases interdependientes pueden encapsularse limpiamente en una clase. La llamada recursiva ordenar dentro del mismo objeto es equivalente a la llamada recursiva en PE. El diseño permite extender fácilmente la clase, por ejemplo sobrescribiendo particion con una estrategia de pivote diferente.

19 PE. Ordenamiento Indirecto Burbuja (Programación Estructurada)

Archivo: 19 ordenamiento de manera indirecta burbuja PE.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: burbuja PE

Descripción:

Se implementa el ordenamiento Burbuja de manera indirecta en PE. El término 'indirecto' se refiere a que el ordenamiento opera sobre structs completos "Persona" en lugar de tipos primitivos, requiriendo el intercambio de estructuras enteras. La función burbuja compara el campo edad de elementos adyacentes e intercambia los structs completos cuando están desordenados. La salida imprime nombre y edad de cada persona.

Técnicas utilizadas:

Ordenamiento indirecto sobre tipo de dato compuesto "struct Persona", intercambio de structs completos con variable temporal del mismo tipo, comparación por campo específico "edad" como criterio de orden, función burbuja con dos ciclos for anidados.

Fragmento clave:

```

void burbuja(Persona arr[], int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++){
            if(arr[j].edad > arr[j+1].edad){
                Persona temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

```

Análisis:

El ordenamiento indirecto amplía el concepto de Burbuja al trabajar con tipos de datos no primitivos. Intercambiar structs completos en lugar de enteros es costoso en memoria si los registros son grandes, lo que motiva el uso de ordenamiento por punteros o por índices en aplicaciones reales. Este programa establece la base conceptual para entender cuándo ordenar datos compuestos es menos eficiente que ordenar referencias a ellos.

19 POO. Ordenamiento Indirecto Burbuja (Programación Orientada a Objetos)

Archivo: 19 ordenamiento de manera indirecta burbuja POO.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: burbuja POO

Descripción:

Se implementa el ordenamiento Burbuja indirecto en POO. La clase Persona declara nombre y edad como atributos públicos. La clase Burbuja encapsula el algoritmo en dos métodos: ordenar, que compara edades de objetos Persona adyacentes e intercambia los objetos completos, y mostrar, que recorre e imprime el resultado. En main se instancia Burbuja y se invocan sus métodos.

Técnicas utilizadas:

Encapsulación del ordenamiento indirecto en clase Burbuja, intercambio de objetos Persona completos dentro de un método de clase, separación de responsabilidades entre ordenar y mostrar, diseño de clase utilitaria de algoritmo que opera sobre arreglos externos.

Fragmento clave:

```
class Burbuja{
public:
    void ordenar(Persona arr[], int n){
        for(int i=0;i<n-1;i++){
            for(int j=0;j<n-1-i;j++){
                if(arr[j].edad > arr[j+1].edad){
                    Persona temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
    }
    void mostrar(Persona arr[], int n){
        for(int i=0;i<n;i++){
            cout<<arr[i].nombre<<" - "<<arr[i].edad<<endl;
        }
    }
};
```

Análisis:

La versión POO del Burbuja indirecto refuerza que el encapsulamiento no depende del tipo de dato que se ordena. La clase Burbuja puede considerarse un algoritmo parametrizable por el tipo de arreglo que recibe. Este patrón es el precursor conceptual de los algoritmos genéricos de la STL (como std::sort) que operan sobre cualquier tipo comparable.

20 PE. Ordenamiento Indirecto Merge Sort (Programación Estructurada)

Archivo: 20 ordenamiento de manera indirecta merge sort PE.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: merge Sort PE

Descripción:

Se aplica el algoritmo Merge Sort de manera indirecta en PE, operando sobre un arreglo de structs Persona. Las funciones merge y mergeSort trabajan con el campo edad como criterio de comparación, pero copian structs completos al arreglo temporal durante la fusión. El resultado

es un arreglo de personas ordenadas ascendentemente por edad, con nombre y edad de cada una impresos al final.

Técnicas utilizadas:

Divide y conquista sobre tipo compuesto "struct Persona", copia de structs completos al arreglo temporal temp[100] durante la fusión, comparación por campo específico como llave de ordenamiento, recursión binaria con índices ini, mid y fin.

Fragmento clave:

```
void merge(Persona arr[], int ini, int mid, int fin){
    Persona temp[100];
    int i=ini, j=mid+1, k=0;
    while(i<=mid && j<=fin){
        if(arr[i].edad < arr[j].edad)
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while(i<=mid) temp[k++] = arr[i++];
    while(j<=fin) temp[k++] = arr[j++];
    for(int x=0; x<k; x++) arr[ini+x] = temp[x];
}
```

Análisis:

El Merge Sort indirecto evidencia el costo adicional de trabajar con tipos compuestos: el arreglo temporal almacena structs completos en lugar de enteros, multiplicando el uso de memoria. A pesar de esto, la complejidad temporal se mantiene en $O(n \log n)$. Este contraste motiva el uso de arreglos de punteros o índices como técnica de ordenamiento indirecto eficiente en registros de gran tamaño.

20 POO. Ordenamiento Indirecto Merge Sort (Programación Orientada a Objetos)

Archivo: 20 ordenamiento de manera indirecta merge sort POO.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: merge Sort POO

Descripción:

El Merge Sort indirecto se encapsula en la clase MergeSort con los métodos merge, ordenar y mostrar. La clase Persona define el tipo de dato. El método ordenar realiza la división recursiva del arreglo de objetos Persona, y merge fusiona los subarreglos copiando objetos completos al arreglo temporal. mostrar imprime el resultado final con nombre y edad de cada persona.

Técnicas utilizadas:

Encapsulación del algoritmo Merge Sort indirecto en clase con tres métodos de responsabilidad única, fusión con copia de objetos completos al arreglo temporal, recursión dentro de método de instancia, diseño de clase de algoritmo separada de la clase de datos.

Fragmento clave:

```
class MergeSort{
public:
    void merge(Persona arr[], int ini, int mid, int fin){
        Persona temp[100]; int i=ini, j=mid+1, k=0;
        while(i<=mid && j<=fin)
            temp[k++] = (arr[i].edad < arr[j].edad) ? arr[i++] : arr[j++];
        while(i<=mid) temp[k++] = arr[i++];
    }
}
```



```

        while(j<=fin) temp[k++] = arr[j++];
        for(int x=0;x<k;x++) arr[ini+x] = temp[x];
    }
    void ordenar(Persona arr[], int ini, int fin){
        if(ini>=fin) return;
        int mid=(ini+fin)/2;
        ordenar(arr,ini,mid); ordenar(arr,mid+1,fin); merge(arr,ini,mid,fin);
    }
};

```

Análisis:

La clase MergeSort para ordenamiento indirecto demuestra que la POO no añade sobrecarga algorítmica: la complejidad temporal y espacial es idéntica a la versión PE. La ganancia es en organización y mantenibilidad. Este programa también cierra el ciclo de comparación PE vs POO para los tres algoritmos del segundo parcial, mostrando que ambos paradigmas son igualmente correctos.

21 PE. Ordenamiento Indirecto Quick Sort (Programación Estructurada)

Archivo: 21 ordenamiento de manera indirecta quick sort PE.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: quick Sort PE

Descripción:

Se aplica el algoritmo Quick Sort de manera indirecta en PE sobre un arreglo de structs Persona, ordenando por el campo edad. La función particion implementa el esquema de Lomuto usando el último elemento como pivote, intercambiando structs Persona completos durante la reorganización. quickSort realiza las llamadas recursivas en los dos sub-arreglos resultantes de la partición.

Técnicas utilizadas:

Esquema de Lomuto con intercambio de structs completos, índice i como marcador de frontera entre elementos menores y mayores al pivote, recursión en sub-arreglos sin arreglo auxiliar (ordenamiento in-place), comparación por campo edad del struct.

Fragmento clave:

```

int particion(Persona arr[], int ini, int fin){
    int pivote = arr[fin].edad;
    int i = ini - 1;
    for(int j=ini;j<fin;j++){
        if(arr[j].edad < pivote){
            i++;
            Persona temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    Persona temp = arr[i+1]; arr[i+1] = arr[fin]; arr[fin] = temp;
    return i+1;
}

```

Análisis:

El Quick Sort indirecto en PE mantiene la ventaja de operar in-place (sin arreglo temporal), lo que lo hace más eficiente en memoria que el Merge Sort indirecto aunque comparten la misma complejidad temporal promedio $O(n \log n)$. Intercambiar structs completos es más costoso que intercambiar enteros o punteros, pero el algoritmo sigue siendo más práctico que Burbuja para conjuntos de datos medianos.

21 POO. Ordenamiento Indirecto Quick Sort (Programación Orientada a Objetos)

Archivo: 21 ordenamiento de manera indirecta quick sort POO.cpp

Concepto principal: Algoritmos de ordenamiento indirecto: quick Sort POO

Descripción:

El Quick Sort indirecto se encapsula en la clase QuickSort con tres métodos: particion, que aplica el esquema de Lomuto intercambiando objetos Persona completos; ordenar, que realiza la recursión sobre los sub-arreglos; y mostrar, que imprime el resultado final. En main se instancia QuickSort y se invoca ordenar con índices 0 y n-1.

Técnicas utilizadas:

Encapsulación del Quick Sort indirecto en clase con tres métodos de responsabilidad única, partición con intercambio de objetos completos, recursión de método de instancia sobre sub-arreglos, diseño que separa la clase de algoritmo "QuickSort" de la clase de datos (Persona).

Fragmento clave:

```
class QuickSort{
public:
    int particion(Persona arr[], int ini, int fin){
        int pivote = arr[fin].edad; int i = ini - 1;
        for(int j=ini;j<fin;j++){
            if(arr[j].edad < pivote){ i++; Persona temp=arr[i]; arr[i]=arr[j];
arr[j]=temp; }
            Persona temp=arr[i+1]; arr[i+1]=arr[fin]; arr[fin]=temp;
            return i+1;
        }
        void ordenar(Persona arr[], int ini, int fin){
            if(ini < fin){ int pi=particion(arr,ini,fin); ordenar(arr,ini,pi-1);
ordenar(arr,pi+1,fin); }
        }
        void mostrar(Persona arr[], int n){
            for(int i=0;i<n;i++) cout<<arr[i].nombre<<" - "<<arr[i].edad<<endl;
        }
    };
};
```

Análisis:

Este programa cierra el bloque de ordenamiento del segundo parcial. La clase QuickSort para datos indirectos integra los conceptos de encapsulamiento, recursividad e intercambio de tipos compuestos. Al comparar las seis implementaciones de ordenamiento (3 algoritmos × 2 paradigmas), se observa que POO agrega organización y cohesión sin penalizar la complejidad algorítmica.

1. Punteros para el Manejo Indirecto con Clases y Estructuras

Archivo: 1 punteros manejo indirecto.cpp

Concepto principal: Punteros: manejo indirecto con clases

Descripción:

Se define la clase Numero con el atributo público valor de tipo entero. En main se crea un objeto n de la clase Numero y un puntero ptr de tipo Numero* que apunta a la dirección de n. El usuario

ingresa un valor que se almacena mediante `ptr->valor` y el programa lo muestra usando el mismo puntero, demostrando el acceso indirecto a atributos de un objeto a través de un puntero.

Técnicas utilizadas:

Declaración de puntero a objeto de clase “Clase `*ptr = &objeto`”, operador de dirección “&” para obtener la dirección de un objeto, operador flecha “->” para acceder a atributos y métodos a través de un puntero, manejo indirecto vs. directo de objetos.

Fragmento clave:

```
class Numero{
public:
    int valor;
};

int main(){
    Numero n;
    Numero *ptr = &n;
    cout<<"Ingresa un numero: ";
    cin>>ptr->valor;
    cout<<"Valor usando puntero: "<<ptr->valor<<endl;
}
```

Análisis:

El manejo indirecto mediante punteros es la base de las estructuras de datos dinámicas en C++. El operador flecha “->” es sintaxis equivalente a `(*ptr).valor`, combinando desreferenciación y acceso a miembro en una sola operación. Este programa sienta las bases para los programas 6, 7 y 7bis donde los punteros se usan para enlazar nodos de pilas, colas y listas dinámicas.

3. PILA: Manejo Estático por Medio de un Arreglo

Archivo: *2 pila estatica.cpp*

Concepto principal: Estructuras de datos: pila estática con arreglo

Descripción:

Se implementa la estructura de datos Pila usando un arreglo estático de 5 enteros dentro de la clase Pila. El atributo privado `tope` rastrea la posición del elemento superior. El constructor inicializa `tope` en -1 (pila vacía). El método `push` inserta un elemento si hay espacio disponible (`tope < 4`). El método `pop` elimina el elemento superior si la pila no está vacía. mostrar recorre desde `tope` hasta 0.

Técnicas utilizadas:

Arreglo estático de tamaño fijo como estructura de almacenamiento, índice `tope` para controlar la posición del último elemento, principio LIFO (Last In, First Out), encapsulamiento con atributos privados y métodos públicos, verificación de capacidad y vacío antes de operar.

Fragmento clave:

```
class Pila{
    int arr[5];
    int tope;
public:
    Pila(){ tope = -1; }
    void push(int x){
        if(tope < 4) arr[++tope] = x;
    }
    void pop(){
```

```

        if(tope >= 0) tope--;
    }
    void mostrar(){
        for(int i=tope;i>=0;i--) cout<<arr[i]<<endl;
    }
};

```

Análisis:

La pila estática con arreglo es la implementación más simple y eficiente en tiempo de acceso: push() y pop() son $O(1)$. Su principal limitación es el tamaño fijo, que puede causar desbordamiento (stack overflow de estructura, no de programa) si se intenta insertar más de 5 elementos. Esta restricción motiva la implementación dinámica del programa 6, donde el tamaño crece según se necesite.

4. COLA: Manejo Estático por Medio de un Arreglo

Archivo: 3 cola_estatica.cpp

Concepto principal: Estructuras de datos: cola estática con arreglo

Descripción:

Se implementa la estructura de datos Cola usando un arreglo estático de 5 enteros dentro de la clase Cola. Los atributos privados frente y fin controlan los extremos de la cola. El constructor inicializa frente en 0 y fin en -1. El método enqueue agrega al final si hay espacio disponible ($\text{fin} < 4$). El método mostrar() recorre el arreglo desde frente hasta fin, respetando el orden de llegada.

Técnicas utilizadas:

Arreglo estático con dos índices (frente y fin) para controlar los extremos de la cola, principio FIFO (First In, First Out), incremento de fin antes de insertar pre-incremento ++fin, recorrido desde frente hasta fin para visualización en orden de llegada.

Fragmento clave:

```

class Cola{
    int arr[5];
    int frente, fin;
public:
    Cola(){ frente=0; fin=-1; }
    void enqueue(int x){
        if(fin < 4) arr[++fin] = x;
    }
    void mostrar(){
        for(int i=frente;i<=fin;i++) cout<<arr[i]<<endl;
    }
};

```

Análisis:

La cola estática con arreglo simple tiene una limitación conocida: una vez que fin llega al índice máximo, no se puede insertar aunque se hayan eliminado elementos del frente (espacio desperdiciado). La solución profesional es la cola circular (circular buffer), donde fin e inicio avanzan módulo n. Esta implementación lineal sirve como punto de partida antes de abordar la versión dinámica del programa 7.

5. LISTA: Manejo Estático por Medio de un Arreglo

Archivo: 4 lista estatica.cpp

Concepto principal: Estructuras de datos: lista estática con arreglo

Descripción:

Se implementa una lista lineal estática con la clase Lista, que almacena hasta 10 enteros en un arreglo privado. El atributo n guarda la cantidad de elementos ingresados. El método llenar solicita la cantidad de elementos al usuario (con mensaje 'Cantidad:') y los lee con cin. El método mostrar recorre el arreglo desde la posición 0 hasta n-1, imprimiendo cada elemento separado por espacio.

Técnicas utilizadas:

Arreglo estático de tamaño fijo (10 elementos) como lista lineal, atributo de tamaño n para controlar el espacio usado, separación de métodos de entrada (llenar) y salida (mostrar), lectura con cin dentro de la clase, impresión en línea con separador de espacio.

Fragmento clave:

```
class Lista{
    int arr[10];
    int n;
public:
    void llenar(){
        cout<<"Cantidad: "; cin>>n;
        for(int i=0;i<n;i++) cin>>arr[i];
    }
    void mostrar(){
        for(int i=0;i<n;i++) cout<<arr[i]<<" ";
    }
};
```

Análisis:

La lista estática con arreglo ofrece acceso $O(1)$ a cualquier elemento por índice, pero con capacidad máxima predefinida. A diferencia de Pila y Cola, la lista no impone un orden de acceso (LIFO o FIFO): se puede acceder a cualquier posición directamente. Esta flexibilidad la hace más versátil, y su limitación de tamaño fijo es la motivación principal para la lista dinámica del programa 7bis.

6. PILA: Manejo Dinámico con Punteros al Mismo Tipo de Dato

Archivo: 5 pila dinamica con puntero.cpp

Concepto principal: Estructuras de datos: pila dinámica con nodos enlazados

Descripción:

Se implementa la Pila de manera dinámica usando memoria heap. La clase Nodo contiene un entero dato y un puntero `Nodo *sig` al siguiente nodo. La clase Pila mantiene un puntero tope al nodo superior. El constructor inicializa tope en NULL. push crea un nuevo nodo con new, lo enlaza al tope actual y actualiza tope. mostrar recorre la cadena de nodos mediante un puntero auxiliar hasta llegar a NULL.

Técnicas utilizadas:

Asignación dinámica de memoria con new, puntero al mismo tipo de dato (Nodo *sig) para enlazar nodos, inicialización de puntero a NULL para indicar lista vacía, puntero auxiliar para recorrido sin modificar tope, principio LIFO con inserción y lectura desde el mismo extremo.

Fragmento clave:

```
class Pila{
    Nodo *tope;
public:
    Pila(){ tope = NULL; }
    void push(int x){
        Nodo *nuevo = new Nodo();
        nuevo->dato = x;
        nuevo->sig = tope;
        tope = nuevo;
    }
    void mostrar(){
        Nodo *aux = tope;
        while(aux){ cout<<aux->dato<<endl; aux = aux->sig; }
    }
};
```

Análisis:

La pila dinámica elimina la restricción de capacidad de la versión estática: puede crecer mientras haya memoria disponible en el heap. El puntero sig de cada Nodo apunta al mismo tipo (Nodo*), lo que constituye una estructura auto-referencial fundamental en C++. La ausencia de delete en este programa implica una fuga de memoria (memory leak), un aspecto a considerar en implementaciones de producción.

7. COLA: Manejo Dinámico con Punteros al Mismo Tipo de Dato

Archivo: 6 cola dinamica con puntero.cpp

Concepto principal: Estructuras de datos: cola dinámica con nodos enlazados

Descripción:

Se implementa la Cola de manera dinámica usando dos punteros: frente (al primer elemento) y fin (al último). El constructor inicializa ambos en NULL. enqueue crea un nodo nuevo con sig=NULL; si la cola está vacía, frente y fin apuntan al mismo nodo; si no, enlaza el nuevo nodo al final (fin->sig = nuevo) y actualiza fin. mostrar recorre desde frente hasta NULL usando un puntero auxiliar.

Técnicas utilizadas:

Dos punteros (frente y fin) para gestionar los extremos de la cola, caso especial de cola vacía (frente y fin = NULL), inserción al final mediante actualización de fin->sig y fin, principio FIFO con inserción al final y acceso desde el frente, puntero auxiliar para recorrido.

Fragmento clave:

```
void enqueue(int x){
    Nodo *nuevo = new Nodo();
    nuevo->dato = x;
    nuevo->sig = NULL;
    if(!frente)
        frente = fin = nuevo;
    else{
        fin->sig = nuevo;
        fin = nuevo;
    }
}
```

```
}
```

Análisis:

La cola dinámica requiere dos punteros (frente y fin) para garantizar inserción $O(1)$ al final y acceso $O(1)$ desde el inicio. Con un solo puntero al frente, insertar al final requeriría recorrer toda la lista ($O(n)$). El manejo del caso vacío (frente == NULL) es crítico: sin esta verificación, fin->sig causaría un acceso a puntero nulo. Este patrón se extiende directamente a colas de prioridad y listas doblemente enlazadas.

7 bis. LISTA: Manejo Dinámico con Punteros al Mismo Tipo de Dato

Archivo: 7 lista dinamica con puntero.cpp

Concepto principal: Estructuras de datos: lista dinámica con nodos enlazados

Descripción:

Se implementa una lista enlazada simple dinámica con inserción al inicio. La clase Nodo almacena un dato entero y un puntero `Nodo *sig`. La clase Lista mantiene un puntero inicio inicializado en NULL. El método insertar crea un nodo nuevo con `new`, lo enlaza al inicio actual (nuevo->sig = inicio) y actualiza inicio al nuevo nodo. mostrar recorre desde inicio hasta NULL usando un puntero auxiliar.

Técnicas utilizadas:

Lista enlazada simple con inserción al frente ($O(1)$), puntero inicio como cabeza de la lista, enlace del nuevo nodo al inicio existente antes de actualizar el puntero de cabeza, recorrido mediante puntero auxiliar que avanza con `aux = aux->sig`, estructura auto-referencial Nodo con puntero al mismo tipo.

Fragmento clave:

```
void insertar(int x){
    Nodo *nuevo = new Nodo();
    nuevo->dato = x;
    nuevo->sig = inicio;
    inicio = nuevo;
}
```

Análisis:

La lista dinámica con inserción al frente es la implementación de lista enlazada más sencilla y eficiente: insertar es $O(1)$. Como efecto secundario, los elementos se muestran en orden inverso al de inserción (el último en entrar aparece primero). Este programa completa la triada de estructuras dinámicas (Pila, Cola, Lista) y sienta las bases para estructuras más complejas como listas doblemente enlazadas, árboles binarios y grafos.

8. PILA: Manejo Dinámico con Librería del IDE (stack)

Archivo: 8 pila libreria.cpp

Concepto principal: Estructuras de datos: pila con STL (stack)

Descripción:

Se implementa una Pila de enteros usando el contenedor stack de la librería estándar de C++ (STL). Se incluye la cabecera <stack>. En main se declara stack<int> pila y se insertan n elementos con push. Para mostrar los elementos en orden LIFO, se consulta el tope con top se imprime y se elimina con pop dentro de un ciclo while que verifica !pila.empty.

Técnicas utilizadas:

Contenedor stack<T> de la STL con tipo genérico, método push() para inserción, top para consulta sin eliminación, pop para eliminación, empty para verificación de vacío, ciclo while con condición de terminación basada en vacío de contenedor.

Fragmento clave:

```
#include <stack>
using namespace std;

int main(){
    stack<int> pila;
    int n,x;
    cin>>n;
    for(int i=0;i<n;i++){ cin>>x; pila.push(x); }
    while(!pila.empty()){
        cout<<pila.top()<<endl;
        pila.pop();
    }
}
```

Análisis:

El contenedor stack de la STL abstrae completamente la implementación interna (que por defecto usa deque), eliminando la necesidad de gestionar memoria manualmente. La interfaz es idéntica conceptualmente a las implementaciones de los programas 3 y 6, pero sin código de estructura. Este programa introduce el paradigma de reutilización de componentes de la STL, fundamental en el desarrollo profesional en C++.

9. COLA: Manejo Dinámico con Librería del IDE (queue)

Archivo: 9 cola libreria.cpp

Concepto principal: Estructuras de datos: cola con STL (queue)

Descripción:

Se implementa una Cola de enteros usando el contenedor queue de la librería estándar de C++ (STL). Se incluye la cabecera <queue>. En main se declara queue<int> cola y se insertan n elementos con push. Para mostrar los elementos en orden FIFO, se consulta el frente con front, se imprime y se elimina con pop dentro de un ciclo while que verifica !cola.empty.

Técnicas utilizadas:

Contenedor queue<T> de la STL, método push para inserción al final, front para consulta del primer elemento sin eliminación, pop para eliminación del frente, empty para verificación de vacío, comportamiento FIFO garantizado por el contenedor.

Fragmento clave:

```
#include <queue>
using namespace std;

int main(){
    queue<int> cola;
```



```

int n,x;
cin>>n;
for(int i=0;i<n;i++){ cin>>x; cola.push(x); }
while(!cola.empty()){
    cout<<cola.front()<<endl;
    cola.pop();
}
}

```

Análisis:

El contenedor queue de la STL implementa internamente una deque (double-ended queue) que garantiza inserción $O(1)$ al final y eliminación $O(1)$ al frente. Comparado con la implementación dinámica del programa 7, queue elimina la necesidad de gestionar los punteros frente y fin manualmente. La interfaz unificada de la STL permite cambiar fácilmente de estructura (de queue a priority_queue, por ejemplo) con cambios mínimos en el código.

10. LISTA: Manejo Dinámico con Librería del IDE (list)

Archivo: 10 lista libreria.cpp

Concepto principal: Estructuras de datos: lista con STL (list)

Descripción:

Se implementa una Lista de enteros usando el contenedor list de la librería estándar de C++ (STL). Se incluye la cabecera <list>. En main se declara list<int> lista. Se leen n elementos y cada uno se inserta al final con push_back. La visualización usa el ciclo for basado en rango (for(int v : lista)) que recorre automáticamente todos los elementos desde el inicio hasta el final.

Técnicas utilizadas:

Contenedor list<T> de la STL (lista doblemente enlazada), método push_back para inserción al final en $O(1)$, ciclo for basado en rango (range-based for, C++11) para recorrido automático, iteración sin índice explícito sobre contenedores STL.

Fragmento clave:

```

#include <list>
using namespace std;

int main(){
    list<int> lista;
    int n,x;
    cin>>n;
    for(int i=0;i<n;i++){ cin>>x; lista.push_back(x); }
    for(int v : lista){
        cout<<v<<" ";
    }
}

```

Análisis:

El contenedor list de la STL implementa internamente una lista doblemente enlazada, permitiendo inserción y eliminación $O(1)$ en cualquier posición conocida. El ciclo for basado en rango es una característica de C++11 que simplifica el recorrido de contenedores, haciendo el código más legible. Comparado con los programas 4 y 7bis, list de la STL es la solución más completa y robusta para listas dinámicas en C++ profesional.